

บทที่ 3

ภาษาจาวา

3.1 ประวัติของภาษาจาวา

เมื่อหลายปีก่อนคงมีหลายคนเชื่อว่าภาษาคอมพิวเตอร์สำหรับโปรแกรมเป็นสิ่งที่ตายแล้ว ซึ่งอาจมีสาเหตุมาจากความเชื่อว่ามีอะไรที่ภาษาซีทำไม่ได้ และอาจต้องใช้เวลามากหลายปีที่คนส่วนใหญ่จะเข้าใจถึงคุณค่าของภาษาซีพลัสพลัส แต่เมื่อปี ค.ศ.1996 ภาษาจาวาก็มีชื่อขึ้นหน้าปกของวารสารคอมพิวเตอร์ชั้นนำเกือบทุกฉบับทั่วโลกและมีการเสนอข่าวออกทาง CNN อย่างต่อเนื่อง ซึ่งไม่เคยมีภาษาคอมพิวเตอร์ใดได้รับความสนใจมากเช่นนี้มาก่อน

ภาษาจาวาเริ่มเป็นที่สนใจมากยิ่งขึ้น เมื่อบริษัท ซัน ไมโครซิสเต็มส์ (Sun Microsystems) ประกาศให้เป็นภาษาสำหรับสร้างโปรแกรมเพื่อใช้งานอินเทอร์เน็ต โปรแกรมที่เขียนโดยภาษาจาวาจะถูกสร้างขึ้นบนคอมพิวเตอร์เครื่องหนึ่ง แล้วนำไปทำงานบนเครื่องคอมพิวเตอร์ต่างระบบปฏิบัติการได้โดยไม่ต้องคอมไพล์โปรแกรมนั้นใหม่ ความสามารถนี้จะเปลี่ยนแปลงบทบาทของอินเทอร์เน็ตไปอย่างสิ้นเชิง เมื่อเรามีโปรแกรมที่สามารถทำงานบนเครื่องคอมพิวเตอร์ใดก็ได้ อินเทอร์เน็ตจะกลายเป็นหนึ่งเดียว ไม่จำเป็นต้องแบ่งแยกสิ่งที่ถูกส่งออกไปมาบนอินเทอร์เน็ตอีกต่อไป เว็บเพจทั้งหลายจะไม่ใช่อเอกสารที่ถูกกระทำแต่จะกลายเป็นเอกสารที่สามารถทำงานได้ นอกไปจากนี้ภาษาจาวาจะส่งผลกระทบต่อทั้งผู้ผลิตและผู้ใช้คือ ถึงเวลาที่บริษัทผู้ผลิตซอฟต์แวร์จะต้องเปลี่ยนแปลงหรือยุบแผนกวินโดวส์, แมคอินทอชหรือยูนิกซ์ให้มารวมกัน เพื่อผลิตโปรแกรมที่ทำงานบนระบบปฏิบัติการใดก็ได้เพียงหนึ่งเดียว สำหรับผู้ใช้งานเมื่อเปิดเครื่องคอมพิวเตอร์เข้าสู่บราวเซอร์ เราสามารถทำการเรียกโปรแกรมที่ต้องการจากเซิร์ฟเวอร์ที่ให้บริการในอินเทอร์เน็ตมาใช้งาน รูปแบบการทำงานของเราก็จะไม่เหมือนเดิมอีกต่อไปคือไม่จำเป็นต้องมีฮาร์ดดิสก์หรือระบบปฏิบัติการอย่างวินโดวส์เลยก็ได้

ผลกระทบเหล่านี้ทำให้ภาษาจาวาเป็นที่น่าสนใจอย่างยิ่ง ในขณะที่บริษัทผู้ผลิตซอฟต์แวร์ทั้งหลายก็ให้การตอบรับภาษาจาวาในทิศทางที่ดีเกินความคาดหมาย ประกอบด้วยภาษาจาวามีประสิทธิภาพในการสร้างโปรแกรมอย่างยิ่ง จึงเป็นเครื่องมือที่ดีสำหรับการสร้างโปรแกรมด้วยคอมไพเลอร์และวิซวลโปรแกรมมิ่ง เชื่อได้แน่นอนว่าในอนาคตอันใกล้อุตสาหกรรมซอฟต์แวร์จะเปลี่ยนโฉมหน้าไปจากเดิม และการเรียนรู้ภาษาจาวาจะเป็นเรื่องที่ได้เปรียบและหลีกเลี่ยงไม่ได้

ในช่วงต้นทศวรรษ 1990 ตลาดเครื่องใช้ไฟฟ้าเช่น เครื่องซักผ้า, หม้อหุงข้าว, โทรทัศน์, ไมโครเวฟและอื่นๆ มีมูลค่าสูงกว่าตลาดคอมพิวเตอร์หลายเท่าตัว อีกทั้งระบบคอมพิวเตอร์ขนาดเล็กสำหรับควบคุมเครื่องใช้ไฟฟ้าเหล่านี้ก็ถูกพัฒนาขึ้นเรื่อยๆ บริษัท ซัน ไมโครซิสเต็มส์ ซึ่งประสบความสำเร็จในตลาดระบบเครือข่ายคอมพิวเตอร์อย่างมากในตอนนั้น เห็นว่าควรใช้ความได้เปรียบของบริษัทเร่งพัฒนาเทคโนโลยีเพื่อยึดครองตลาดเครื่องใช้ไฟฟ้าที่มีคอมพิวเตอร์ควบคุมไว้ก่อนคนอื่น จึงจัดทีม Green group ขึ้นในปี 1991 มี James Gosling เป็นหัวหน้าสำหรับพัฒนาระบบซอฟต์แวร์ควบคุม

เครื่องใช้ไฟฟ้าขนาดเล็ก ทีม Green group ได้สร้างเครื่องต้นแบบที่เรียกว่า Star 7 เป็นระบบบริโหมทคอนโทรลขนาดมือถือ สามารถควบคุมการเคลื่อนไหวของวัตถุโดยใช้นิ้วสัมผัสบนแป้นแสดงผล ระบบนี้ถูกทดสอบใช้แสดงการควบคุมตัว The Dunk (ซึ่งต่อมาเป็นตัวนำโชคของภาษาจาวา) ใช้เคลื่อนที่ผ่านกลุ่มของวัตถุในมิติสมมติ

Green group ใช้ภาษาซีพลัสพลัสเขียนโปรแกรมของ Star 7 ผลที่ได้เป็นโปรแกรมที่มักจะทำงานได้ผิดพลาดและล้มเหลวบ่อยๆ จนต้องสรุปว่าภาษาซีพลัสพลัสไม่เหมาะสำหรับงานแบบนี้ เพราะมีข้อจำกัดหลายอย่างนั่นคือ เครื่องใช้ไฟฟ้าขนาดเล็กจะมีหน่วยความจำน้อย ไม่เหมาะกับการเขียนโปรแกรมด้วยภาษาซีพลัสพลัสซึ่งมักจะมีขนาดใหญ่ อีกทั้งในเครื่องใช้ไฟฟ้าเหล่านี้ไม่มีระบบปฏิบัติการโปรแกรมจึงไม่สามารถเรียกใช้บริการของระบบปฏิบัติการได้เหมือนบนเครื่องคอมพิวเตอร์ซึ่งมีระบบปฏิบัติการ ดังนั้นความสามารถของภาษาซีพลัสพลัสจึงถูกจำกัดไปอย่างมาก เราจึงต้องสร้างโปรแกรมสำหรับทำงานพื้นฐานเอง นอกจากนี้ภาษาซีพลัสพลัสยังเป็นภาษาที่ไม่ปลอดภัย เพราะยอมให้มีการใช้พอยน์เตอร์อย่างไม่จำกัด และละเลยการทำการตรวจสอบชนิดข้อมูลโปรแกรมจึงมักมีความผิดพลาดซ่อนอยู่เป็นจำนวนมาก

ปัญหาที่สำคัญกว่าคือ หน่วยประมวลผลที่ใช้ในงานควบคุมมีมากหลายเบอร์หลายยี่ห้อ และมีชุดคำสั่งที่แตกต่างกัน โปรแกรมที่ทำงานได้บนหน่วยประมวลผลรุ่นหนึ่งจะต้องถูกคอมไพล์ใหม่ จึงจะนำไปใช้งานบนหน่วยประมวลผลอีกรุ่นหนึ่งได้ ด้วยเหตุนี้พวกเขาจึงพัฒนาภาษาใหม่ชื่อโอ๊กให้เป็นภาษาที่ง่ายต่อการเรียนรู้และใช้งาน ไม่มีข้อผิดพลาดในตอนทำงานและเหมาะที่จะทำงานในระบบที่มีหน่วยความจำน้อย พวกเขาแน่ใจว่าระบบควบคุมจะมีขนาดใหญ่และซับซ้อนขึ้นเรื่อยๆ จึงให้โอ๊กเป็นภาษาเชิงวัตถุเพื่อให้ง่ายต่อการสร้างและดูแลระบบขนาดใหญ่ ปัญหาใหญ่ของการออกแบบภาษาโอ๊กอยู่ที่ความต้องการให้เป็นภาษาที่ทำงานบนหน่วยประมวลผลใดก็ได้ จึงนำเทคนิคการคอมไพล์โปรแกรมเป็นคำสั่งของหน่วยประมวลผลสมมติตัวหนึ่งแล้วสร้างอินเทอร์พรีเตอร์ของหน่วยประมวลผลสมมติตัวนั้นให้แก่หน่วยประมวลผลที่ทำงานที่จะทำงานโปรแกรมนั้น ด้วยวิธีนี้โปรแกรมที่สร้างขึ้นจึงสามารถนำไปทำงานบนเครื่องที่มีหน่วยประมวลผลต่างรุ่นได้ ซึ่งเรียกคุณสมบัตินี้ว่า Platform independent

ผลงานของ Green group ให้ความหวังแก่บริษัท ชัน ไมโครซิสเต็มส์ อย่างมาก จึงก่อตั้งบริษัทลูกชื่อว่า FirstPerson Inc. ในปี ค.ศ.1992 เพื่อร่วมมือกับบริษัท Time Warner พัฒนาระบบ video on demand ที่มีอุปกรณ์ควบคุมเครื่องรับโทรทัศน์ที่สามารถติดต่อกับผู้ใช้ในลักษณะของ Interactive TV ภาษาโอ๊กถูกใช้เขียนโปรแกรมของ set-top box ซึ่งเป็นกล่องที่ใช้ต่อพ่วงกับโทรทัศน์เพื่อควบคุมและติดต่อกับผู้ใช้ แม้ระบบนี้จะถูกพัฒนาและสามารถใช้งานได้ แต่บริษัท Time Warner กลับหยุดโครงการนี้ไป บริษัท ชัน ไมโครซิสเต็มส์ จึงพยายามหาลูกค้ารายใหม่ให้แก่เทคโนโลยีนี้ โดยนำไปทดสอบใช้งานอื่นๆ เช่น ระบบควบคุมบัตรเครดิต ระบบควบคุมเครื่องจักรในโรงงานอุตสาหกรรมและระบบควบคุมเครื่องซีดีรอม แต่ไม่สามารถหาลูกค้าได้

ในปี ค.ศ.1993 ไฮเปอร์เท็กซ์และบราวเซอร์เปลี่ยนแปลงอินเทอร์เน็ตไปอย่างมากและมีผู้ใช้เพิ่มมากขึ้นอย่างรวดเร็ว บริษัท ชัน ไมโครซิสเต็มส์ มองเห็นความจำเป็นที่ต้องมีภาษาสำหรับสำหรับสร้างโปรแกรมซึ่งสามารถทำงานบนคอมพิวเตอร์เครื่องใดก็ได้และเพ่งนึกถึงคุณสมบัติ Platform independent

ของภาษาโอ๊ก จึงนำภาษาโอ๊กมาปรับปรุงใหม่ และทดลองสร้างเว็บเบราว์เซอร์ชื่อเว็บรันเนอร์ซึ่งสามารถทำงานโปรแกรมภาษาโอ๊กได้ เมื่อทดลองจนได้ผลพวกเขาทราบว่ามีความสำคัญอย่างยิ่งในมือแล้ว แต่โอ๊กเป็นชื่อทางการค้าที่มีผู้ใช้อยู่ก่อนจึงเปลี่ยนชื่อใหม่เป็นจาวาในช่วงต้นปี ค.ศ.1995 พร้อมกับเปลี่ยนชื่อเว็บรันเนอร์ใหม่เป็นฮอตจาวา (HotJava)

เมื่อเริ่มต้นจาวาทำงานบนระบบโซลาริสเท่านั้น แต่เพียงภายในฤดูร้อนของปี ค.ศ.1995 ก็พัฒนาให้ทำงานได้บนวินโดวส์ เอ็นที, วินโดวส์ 95 และลินุกซ์พอลถึงปลายปี ค.ศ.1995 บริษัทเน็ตสเคปก็สร้างเน็ตสเคป 2.0 ให้สามารถทำงานร่วมกับภาษาจาวาได้ หลังจากนั้นบริษัทไมโครซอฟท์และไอบีเอ็มก็ประกาศสนับสนุนภาษาจาวาด้วย จนกระทั่งช่วงปลายปี ค.ศ.1995 บริษัท ซัน ไมโครซิสเต็มส์ นำโปรแกรมชุดพัฒนาภาษาจาวา JDK (Java Development Kit) รุ่น 1.0 ขึ้นแจกจ่ายในอินเทอร์เน็ต

3.2 คุณสมบัติของภาษาจาวา

ลักษณะของภาษาจาวาเป็นการเขียนโปรแกรมอ้างอิงเชิงวัตถุ OOP (Object Oriented Programming) ซึ่งถูกออกแบบให้มีลักษณะดังต่อไปนี้

1. ภาษาจาวาเป็นภาษาที่ง่ายต่อการเรียนรู้และนำไปใช้งาน ซึ่งเราสามารถแยกพิจารณาออกตามได้หลายมุมมองดังต่อไปนี้

- ภาษาจาวานำไวยากรณ์ภาษาส่วนใหญ่มาจากภาษาซีและซีพลัสพลัส ซึ่งทำให้ผู้ที่คุ้นเคยกับการเขียนโปรแกรมด้วยภาษาซีและซีพลัสพลัสสามารถเข้าใจภาษาจาวาได้ง่ายหรือใช้เวลาศึกษาไม่นาน
- ภาษาจาวามีกลไกของภาษาไม่มากและไม่ซับซ้อน โดยได้ทำการตัดกลไกของภาษาซีและซีพลัสพลัสที่มีความซับซ้อนเช่น pointer, default argument, scope resolution, protected and private inheritance และ operator overloading แต่ในขณะเดียวกันก็เพิ่มความสามารถให้คอมไพเลอร์ของภาษาจาวา ทำให้ไม่มี preprocessor commands ดังนั้นจะไม่มี macros definitions, included files, conditional compilation และ header files และเมื่อจาวาเป็นภาษาเชิงวัตถุแล้วกลไกอย่างเช่น structures, union, bit fields และ enumerated types รวมทั้งการทำ typedef ก็ไม่มีความจำเป็นจึงถูกตัดออกไป ภาษาจาวาถูกออกแบบให้เป็นภาษาเชิงวัตถุได้รอบคอบกว่าภาษาซีพลัสพลัส ดังจะเห็นได้จากกลไกที่ทำให้เกิดปัญหาเช่น multiple inheritance, copy constructor และกลไกที่อาจทำลายแบบแผนการเขียนโปรแกรมเชิงวัตถุที่คืออย่างเช่น friend methods ก็ถูกตัดออกไปเช่นกัน
- ภาษาจาวาประสบความสำเร็จอย่างมากในการใช้เทคนิคของภาษาเชิงวัตถุ ช่วยให้สร้างโปรแกรมที่ยู่ยากให้ง่ายขึ้น ดังจะเห็นได้ว่าหากเป็นภาษาคอมพิวเตอร์ภาษาอื่นๆ การสร้างโปรแกรมที่เกี่ยวข้องกับการสร้างกราฟิกในส่วนติดต่อกับผู้ใช้งาน (GUIs), multitasking network และ distributed objects ผู้เขียนโปรแกรมจะต้องมีความรู้ในภาษานั้นสูงจึงจะสามารถทำได้ หรือไม่เช่นนั้นก็ต้องใช้โปรแกรมประเภทวิชวลโค้ดอย่างเช่นวิชวลซีพลัสพลัส (Visual C++) ช่วยในการสร้างโปรแกรม แต่ภาษาจาวามีกลไกการเขียนโปรแกรมเชิงวัตถุที่ส่งเสริมการนำโปรแกรมที่

สร้างไว้แล้วมาใช้งานใหม่ ซึ่งสามารถเปลี่ยนแปลงหรือเพิ่มเติมโปรแกรมบางส่วนให้เหมาะกับงานที่ต้องการ โดยไม่จำเป็นต้องทราบรายละเอียดของโปรแกรมเดิมทั้งหมด ทำให้เราสามารถสร้างโปรแกรมสำหรับงานที่ยุ่งยากขึ้นได้โดยใช้บางส่วนจากโปรแกรมที่มีผู้สร้างไว้แล้วโดยง่าย

- ในภาษาที่ถูกเรียกว่า typed language จะทำการตรวจสอบเกี่ยวกับการประกาศชนิดของตัวแปร เพื่อช่วยให้โปรแกรมทำงานโดยไม่มีผลผิดพลาดเกี่ยวกับชนิดของตัวแปร แต่การเขียนโปรแกรมภาษาเช่นนี้มีข้อยุ่งยากเช่น ตัวอักษรบวกกับเลขจำนวนเต็มไม่ได้ เนื่องจากค่าที่จะนำมาทำการคำนวณหรือกำหนดค่าให้แก่มันจะต้องเป็นตัวแปรชนิดเดียวกัน ภาษาจาวาเองก็เป็น typed language แต่เพื่อให้ใช้งานง่ายจึงสร้างกลไกเกี่ยวกับ automatic type correction ซึ่งเป็นการเปลี่ยนแปลงค่าระหว่างชนิดของตัวแปรที่แตกต่างกัน

2. โปรแกรมที่สร้างขึ้นด้วยภาษาจาวาจะไม่มีผลผิดพลาดจากข้อบกพร่องของภาษา นั่นคือโปรแกรมจะต้องไม่ล้มเหลว (fail) ด้วยความผิดพลาดเพียงเล็กน้อยที่ไม่เกี่ยวกับตรรกะของโปรแกรม คุณสมบัติของภาษาลักษณะนี้จะมีความคงทน (robust) ภาษาจาวาถูกออกแบบให้มีความคงทนตามลักษณะดังนี้

- ภาษาจาวาเน้นการใช้กลไก exception handling เพื่อให้โปรแกรมสามารถจัดการกับความผิดปกติบางอย่างที่เกิดขึ้นในขณะที่โปรแกรมทำงาน ซึ่งโปรแกรมจะทำงานต่อไปได้โดยไม่หยุดลง
- ภาษาจาวาตัดกลไกบางอย่างในภาษาซีและซีพลัสพลัสที่อาจทำให้เกิดความผิดพลาด หากใช้อย่างไม่ระมัดระวังเช่น global variable, variable length arguments และ goto statement นอกจากนี้ภาษาจาวายังได้ตัดการอ้างถึงแอดเดรสของตัวแปร และการใช้พอยน์เตอร์สำหรับอ่านหรือเขียนข้อมูลลงในหน่วยความจำโดยตรง
- ภาษาจาวาไม่มีกลไกคืนหน่วยความจำ (de-allocation) ที่ขอมาในขณะที่โปรแกรมทำงานอย่างที่มีใช้งานในภาษาซีและซีพลัสพลัสคือ free() และ delete() ภาษาจาวาอาศัย automatic garbage collector ทำหน้าที่เก็บหน่วยความจำที่ไม่สามารถอ้างถึงได้แล้วกลับไปใช้งานใหม่
- ภาษาจาวาเป็นภาษาประเภท Strongly typed หมายถึงภาษาที่เน้นความถูกต้องของชนิดข้อมูลที่ใช้ในโปรแกรม คอมไพเลอร์ของภาษาประเภทนี้จะทำการตรวจสอบว่าโปรแกรมการจัดการกับชนิดข้อมูลของตัวแปรถูกต้องหรือไม่เรียกการทำงานนี้ว่า type checking ซึ่งความผิดพลาดเกี่ยวกับชนิดข้อมูลทั้งหมดจะถูกปฏิเสธตั้งแต่ตอนคอมไพล์โปรแกรม จึงไม่มีความผิดพลาดเกี่ยวกับชนิดข้อมูลเกิดขึ้นในระหว่างที่โปรแกรมทำงาน นอกจากนั้นยังมีการตรวจสอบอีกว่าในระหว่างที่โปรแกรมทำงานมีการเปลี่ยนค่าตัวแปรในชนิดข้อมูลหนึ่งไปเป็นค่าในอีกชนิดข้อมูลหนึ่งถูกต้องหรือไม่ รวมทั้งการอ้างถึงสมาชิกในอาร์เรย์ (array) หรือสตริง (String) อยู่ในขอบเขตที่ถูกต้องหรือไม่

3. โปรแกรมภาษาจาวามักจะถูกส่งผ่านระบบเครือข่ายไปทำงานบนเครื่องคอมพิวเตอร์ของผู้อื่น ดังนั้นภาษาจาวาจึงต้องมีหลักประกันให้แก่ผู้รับโปรแกรมนั้นไปทำงาน เพื่อจะไม่ก่อให้เกิดอันตรายและความเสียหายต่อเครื่องหรือระบบของผู้ใช้นั้น ทำให้ภาษาจาวาจึงต้องมีข้อกำหนดหลายอย่างเพื่อให้

โปรแกรมไม่สามารถทำอันตรายหรือสร้างความเสียหายให้กับระบบที่รับโปรแกรมนั้นไปทำงาน คุณสมบัติในลักษณะคือความปลอดภัย (Security) แต่อย่างไรก็ตามไม่มีภาษาคอมพิวเตอร์ใดที่มีความปลอดภัยครบถ้วน ภาษาจาวาถูกจัดว่ามีความปลอดภัยในระดับสูงเท่านั้น เพราะถูกออกแบบมาเพื่อความปลอดภัยมากกว่าภาษาอื่น โดยมีการป้องกันในลักษณะดังนี้

- จากการที่ภาษาจาวาไม่ยอมให้มีการอ้างถึงค่าในหน่วยความจำผ่านทางพอยน์เตอร์ และจะทำการตรวจสอบว่ามีการอ้างถึงสมาชิกในอาร์เรย์อยู่ในขอบเขตหรือไม่ โปรแกรมจึงไม่สามารถเขียนหรืออ่านค่าในหน่วยความจำที่ไม่มีสิทธิ์อ้างถึง การเปลี่ยนแปลงโปรแกรมหรือค่าในหน่วยความจำเพื่อทำการสร้างโปรแกรมที่จะเป็นอันตรายต่อผู้อื่นจึงไม่สามารถทำได้
 - อินเทอร์พรีเตอร์ของภาษาจาวามี byte-code verifier สำหรับทำหน้าที่ตรวจสอบโปรแกรมที่จะถูกทำงานว่ามีคำสั่งที่ผิดปกติหรือมีการทำงานที่ไม่สมควรหรือไม่ หากตรวจพบก็จะทำการปฏิเสธการทำงานในโปรแกรมนั้น
 - ภาษาจาวาจะมีระบบรักษาความปลอดภัยที่เรียกว่า sandbox model นั่นคือถ้าหากเป็นโปรแกรมที่ถูกนำมาจากเครื่องอื่นผ่านทางระบบเครือข่ายจะถือว่าเป็นโปรแกรมที่ไม่น่าไว้วางใจ (un-trust codes) และจะถูกเก็บอยู่ในภาวะที่เรียกว่า sandbox โปรแกรมที่อยู่ใน sandbox จะมีข้อจำกัดในการทำงานหลายอย่าง ซึ่งถูกควบคุมโดย security manager เช่นไม่สามารถอ่านหรือเขียนไฟล์ได้ เป็นต้น
4. คุณสมบัติสำคัญซึ่งถือเป็นจุดมุ่งหมายของการออกแบบภาษาจาวาคือ โปรแกรมต้องสามารถทำงานบนเครื่องต่างระบบกันได้ ซึ่งเราเรียกคุณสมบัติของภาษาในลักษณะนี้ว่า architecture neutral หรือ platform independent ทำให้โปรแกรมภาษาจาวาสามารถทำงานบนเครื่องคอมพิวเตอร์หรือระบบใดก็ได้โดยไม่เกิดความผิดพลาดและได้ผลลัพธ์ออกมาเหมือนกัน

3.3 การใช้ภาษาจาวาสำหรับสร้างและพัฒนาในงานด้านต่างๆ

จากความสามารถของภาษาจาวาดังที่กล่าวผ่านมาข้างต้น คงสามารถสรุปได้ว่าภาษาจาวาเป็นภาษาคอมพิวเตอร์สำหรับโปรแกรมที่มีความสามารถรอบด้าน อีกทั้งยังมีความยืดหยุ่นภายในตัวสูง ดังนั้นการที่จะนำเอาภาษาจาวามาสร้างและพัฒนาในงานด้านต่างๆ ให้มีประสิทธิภาพลงจะทำได้สะดวกขึ้น

ภาษาจาวาสามารถใช้เป็นเครื่องมือในการพัฒนาซอฟต์แวร์ด้านไคลเอ็นต์ได้อย่างดีภาษาหนึ่ง โดยเฉพาะงานด้านกราฟิกและระบบติดต่อกับผู้ใช้ เพราะปัจจุบันโปรแกรมที่เขียนขึ้นจากภาษาจาวาจะทำงานที่ฝั่งไคลเอ็นต์เป็นส่วนมากจะเห็นได้จากจาวาแอปเพล็ตต่างๆ บนเว็บเบราว์เซอร์ แต่จะให้ดีและสมบูรณ์ยิ่งขึ้นต่อความเป็นไคลเอ็นต์ซอฟต์แวร์ที่ดีก็คือ การติดต่อระบบฐานข้อมูลจากไคลเอ็นต์ไปเป็นเซิร์ฟเวอร์ แม้ว่าในปัจจุบันจาวาจะสามารถทำได้แล้วก็ตามแต่ก็จำเป็นต้องใช้เครื่องมือชนิดอื่นเป็นตัวช่วย เช่นการเชื่อมต่อกับฐานข้อมูลแบบ ODBC (Open Database Connectivity) ซึ่งการทำงานของทางฝั่งเซิร์ฟเวอร์ยังคงต้องใช้ภาษาคอมพิวเตอร์ชนิดอื่นช่วยเพื่อการติดต่อกับฐานข้อมูล

หากนำภาษาจาวาไปใช้สร้างโปรแกรมประยุกต์บนเซิร์ฟเวอร์นั้น ภาษาจาวาจำเป็นอย่างยิ่งที่จะต้องได้รับการปรับปรุงการทำงานอยู่เสมออย่างคือ การเชื่อมต่อกับฐานข้อมูลโดยตรง, การทำงานให้สอดคล้องกับอินพุตและเอาต์พุต และความเร็วของการรันต้องเพิ่มขึ้นเรื่อยๆ กับโปรแกรมประยุกต์ทั่วไป โดยระบบ JDBC (Java Database Connectivity) ได้เข้ามาแก้ปัญหาในส่วนของการติดต่อกับฐานข้อมูลโดยตรง และจากการร่วมมือกันของบริษัทซอฟต์แวร์ที่มีชื่อเสียงหลายบริษัทได้สร้างมาตรฐานการเชื่อมต่อกับฐานข้อมูลของจาวาขึ้น ทำให้การเชื่อมต่อฐานข้อมูลจากไคลเอ็นต์มายังเซิร์ฟเวอร์จะทำได้ง่ายขึ้นโดยใช้ภาษาจาวา

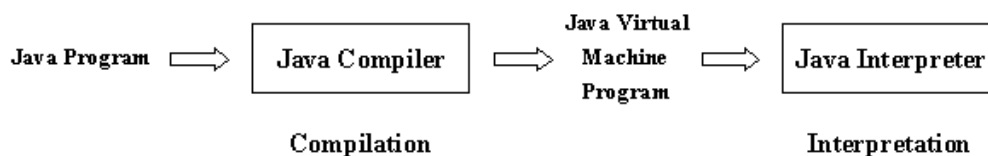
นอกจากนี้ภาษาจาวายังได้สร้างสภาพแวดล้อมเสมือน (Java Virtual Machine) ซึ่งจะอนุญาตให้เฉพาะจาวาแอปเพล็ตและจาวาแอปพลิเคชันซึ่งเป็นไบนารีโค้ดเท่านั้นที่สามารถรันได้ โดยไบนารีโค้ดจะปราศจากไวรัสหรือส่วนที่จะทำอันตรายต่อระบบ เนื่องจากสภาพแวดล้อมเสมือนจะจำกัดสิทธิการเข้าใช้ทรัพยากรของ ไบนารีโค้ด และในปัจจุบันการรักษาความปลอดภัยอีกระบบหนึ่งก็ถูกนำมาปรับใช้คือ การเข้ารหัสข้อมูลเพื่อเพิ่มความปลอดภัยในการรับส่งข้อมูล

การใช้ภาษาจาวาประยุกต์สร้างและพัฒนางานยังสามารถแบ่งออกเป็นประเภทต่างๆ ตามลักษณะของกลุ่มงานได้ดังนี้

- งานด้านการศึกษา โดยจัดทำเป็นลักษณะสั่งการสอนคล้ายกับ CAI มีความสามารถในการโต้ตอบระหว่างผู้ใช้งานกับคอมพิวเตอร์
- สร้างซอฟต์แวร์สำหรับเป็นเครื่องมือใช้พัฒนาโปรแกรม (Software Developer tool kits)
- สร้างแอปพลิเคชันทางธุรกิจให้มีความสามารถในการประมวลผลข้อมูลได้ด้วยตัวเอง
- พัฒนาเกมให้มีความเป็นมัลติเพลเยอร์ สามารถเล่นได้หลายๆ คนพร้อมกันผ่านทางระบบเครือข่ายหลากหลายพื้นที่ไม่ว่าจะเล่นด้วยกัน ณ ตำแหน่งใดทั่วทุกมุมโลก
- ตอบสนองงานทางด้านเว็บเพจทำให้งานสร้างเว็บเพจดูมีชีวิตชีวาและลูกเล่นมากขึ้น โดยการใช้ความสามารถทางด้านมัลติมีเดีย
- สร้างตัวจัดการบนอินเทอร์เน็ต มีความสามารถในการเรียกหาข้อมูลผ่านทางระบบเครือข่าย

3.4 สภาพแวดล้อมเสมือน (Java Virtual Machine)

สภาพแวดล้อมเสมือนเป็นหลักการสร้างคอมพิวเตอร์จำลองของภาษาจาวา โดยจะสมมติให้มีคอมพิวเตอร์อีกเครื่องหนึ่งขึ้นมา โดยคอมพิวเตอร์สมมติเครื่องนี้จะช่วยในการคอมไพล์โปรแกรมภาษาจาวาทุกโปรแกรม เมื่อต้องการให้โปรแกรมภาษาจาวาไปทำงานบนคอมพิวเตอร์จริงๆ เราก็เพียงแค่สร้างตัวอินเทอร์พรีเตอร์ของคอมพิวเตอร์จำลองตัวนี้บนเครื่องคอมพิวเตอร์เครื่องนั้น ภาษาจาวาทุกโปรแกรมก็จะสามารถทำงานบนระบบคอมพิวเตอร์นั้นได้ตามต้องการ ด้วยหลักการนี้ก็เป็นที่มาของคุณสมบัติของจาวาที่ไม่ขึ้นกับแพลตฟอร์มและฮาร์ดแวร์ใดๆ เราอาจจะเรียก Java Virtual Machine สั้นๆ ว่า JVM ก็ได้



รูปที่ 3.1 การทำงานด้วยคอมพิวเตอร์สมมติที่จำลองโดยจาวาอินเทอร์พรีเตอร์

สำหรับเหตุผลที่ต้องมี JVM นี้จริงๆ แล้วเป็นการกำหนดค่าขึ้นมาสำหรับเป็นคำจำกัดความเฉพาะในความคิดเท่านั้น เพื่อให้ นักพัฒนาจะได้ไม่ถูกบังคับให้ต้องสร้างตัวอินเทอร์พรีเตอร์ตามแนวทางใดแนวทางหนึ่งโดยเฉพาะ แต่อินเทอร์พรีเตอร์ที่สร้างตามข้อกำหนดดังกล่าวไม่ว่าจะอยู่บนแพลตฟอร์มใดจะสามารถรันโปรแกรมที่เขียนขึ้นด้วยภาษาจาวาได้โดยให้ผลลัพธ์ออกมาเหมือนกัน โดยบริษัทจาวาซอฟต์แวร์ซึ่งเป็นบริษัทลูกของบริษัท ซัน ไมโครซิสเต็มส์ เป็นผู้กำหนดชุดคำสั่ง JVM รวมทั้งความหมายของแต่ละคำสั่ง ข้อกำหนดเหล่านี้ถือเป็นมาตรฐานของภาษาจาวาที่เผยแพร่ให้แก่บุคคลทั่วไป เพื่อให้ผู้พัฒนาสามารถสร้าง JVM ของตนขึ้นได้ไม่ว่าจะใช้วิธีทางฮาร์ดแวร์หรือซอฟต์แวร์ ภายใน JVM จะมีหน่วยประมวลผลสมมติที่เรียกว่าเวอร์ชวลโปรเซสเซอร์ (virtual processor) ซึ่งทำหน้าที่ประมวลผลคำสั่งของ JVM

ปัจจุบัน JVM ที่เป็นฮาร์ดแวร์ยังอยู่ในระหว่างการพัฒนา ดังนั้น JVM เกือบทั้งหมดที่ใช้งานกันอยู่ในขณะนี้ จึงเป็นโปรแกรมที่จำลองการทำงานของ JVM บนเครื่องคอมพิวเตอร์ทั่วไป โดยปกติแล้วเวอร์ชวลโปรเซสเซอร์ของ JVM ที่จำลองขึ้นบนคอมพิวเตอร์เครื่องหนึ่งจะแปลคำสั่งของ JVM เป็นคำสั่งของหน่วยประมวลผลในคอมพิวเตอร์เครื่องนั้นซึ่งเรียกว่าเนทีฟโค้ด (native code) แล้วให้หน่วยประมวลผลทำงานคำสั่งนั้น

ออปโค้ด (opcode) ของ JVM มีขนาด 1 ไบต์ทุกคำสั่ง เราจึงเรียกโปรแกรม JVM ว่าโปรแกรมไบต์โค้ด (byte code) จำนวนคำสั่งของ JVM มีได้สูงสุดเพียง 256 คำสั่ง เมื่อเปรียบเทียบกับหน่วยประมวลผลทั่วไปแล้วดูคล้ายกับว่าจำนวนคำสั่งของ JVM มีมากมาย ซึ่งที่จริงแล้วคำสั่งของ JVM แบ่งออกเป็นไม่กี่ประเภท โดยแต่ละประเภทจะทำหน้าที่คล้ายๆ กันเพียงแต่ทำกับโอเปอเรนด์ (operands) ต่างชนิดข้อมูลกันเท่านั้น

ชุดคำสั่ง JVM ถูกออกแบบมาเพื่อสนับสนุนการทำงานของโปรแกรมเชิงวัตถุจึงมีคำสั่งเกี่ยวกับการสร้างอินสแตนซ์ (instances) และการอ้างอิงสมาชิกในอินสแตนซ์ซึ่งไม่มีในหน่วยประมวลผลทั่วไป ภาษาจาวาเป็นภาษาที่เน้นความถูกต้องเกี่ยวกับชนิดของข้อมูล จึงมีคำสั่งสำหรับคำนวณชนิดของข้อมูลพื้นฐานแต่ละชนิด ดังเช่นในคำสั่ง iadd สำหรับบวกเลขจำนวนเต็มชนิด integer และคำสั่ง dadd สำหรับบวกเลขทศนิยมชนิด double เป็นต้น ในบางคำสั่งของ JVM จะเหมือนกับคำสั่งที่มีในหน่วยประมวลผลทั่วไป

JVM ถูกออกแบบให้สามารถจำลองได้บนหน่วยประมวลผลทั่วไป แต่หน่วยประมวลผลทั่วไปนั้นมีจำนวนรีจิสเตอร์ไม่เท่ากัน บางรุ่นมีรีจิสเตอร์ที่ทำหน้าที่พิเศษกว่ารุ่นอื่น ผู้ออกแบบจึงจัดปัญหานี้

โดยให้ JVM ไม่มีรีจิสเตอร์และทำการคำนวณทั้งหมดบนสแต็ก (stack) ชุดคำสั่งของ JVM จึงเป็น stacked operations ซึ่งกล่าวได้ว่า JVM เป็น stack machine

เวอร์ชวลไพเรสเซอร์ใน JVM จะเปลี่ยนคำสั่งไบต์โค้ดไปเป็นเนทีฟโค้ดที่ทำหน้าที่เดียวกันแล้วทำงานเนทีฟโค้ดนั้น สังเกตว่าเนทีฟโค้ดนั้นอาจเป็นชุดคำสั่งของระบบปฏิบัติการที่ใช้หรืออาจเป็นไลบรารีมาตรฐาน (standard library) ที่สร้างขึ้นสำหรับหน่วยประมวลผลนั้น ทำให้ JVM หนึ่งอาจใช้งานได้ในระบบต่างๆ โดยเปลี่ยนแปลงแค่ไลบรารีมาตรฐานเท่านั้น

โปรแกรมภาษาจาวาเป็นโปรแกรมที่มีการทำงานโดยการอินเทอร์พรีตชันคือ ต้องทำการแปลภาษาเพื่อให้สามารถใช้งานได้บนทุกๆ แพลตฟอร์ม ซึ่งโปรแกรมที่ทำงานโดยการอินเทอร์พรีตชันย่อมทำงานได้ช้ากว่าโปรแกรมที่ทำงานโดยตรง ทำให้ปัจจุบันโปรแกรมภาษาจาวาจะทำงานช้ากว่าภาษาซีประมาณ 10 เท่า ได้มีการค้นคว้าเพื่อเพิ่มความเร็วของภาษาจาวาเช่น สร้างหน่วยประมวลผลที่สามารถทำงานคำสั่งของ JVM ได้โดยตรงและพัฒนาจาวาอินเทอร์พรีเตอร์ซึ่งไม่ได้ทำงานคำสั่ง JVM ทีละคำสั่งแต่จะเปลี่ยนโปรแกรม JVM ทั้งโปรแกรมให้เป็นเนทีฟโค้ดแล้วทำงานตามโค้ดนั้น วิธีการนี้เรียกว่า Just In Time (JIT) เพราะคล้ายกับว่าทำการคอมไพล์โปรแกรม JVM ก่อนจะเริ่มทำงาน ซึ่งโปรแกรมจะทำงานเร็วขึ้นมากเพราะคำสั่งที่ถูกทำซ้ำบ่อยๆ เช่น คำสั่งที่อยู่ในประโยคทำซ้ำหรือฟังก์ชันที่ถูกเรียกใช้บ่อยๆ จะถูกแปลเพียงครั้งเดียวไม่ใช่ทุกครั้งที่ถูกนำไปใช้งาน

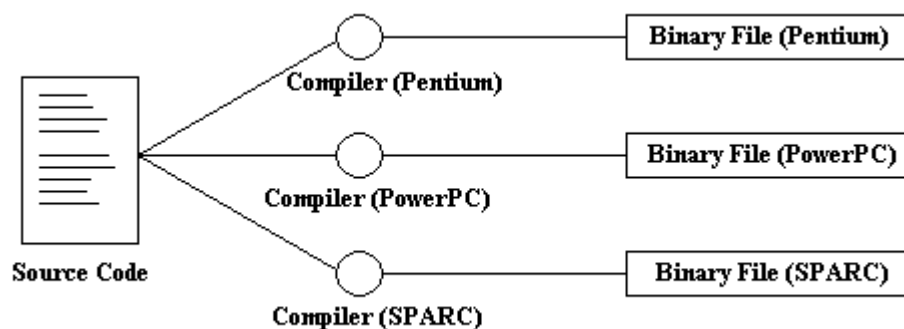
3.5 หลักการทำงานของภาษาจาวา

ในที่นี้ขออธิบายหลักการทำงานของภาษาจาวา โดยเริ่มตั้งแต่ทำการคอมไพล์ตัวโปรแกรมจนกระทั่งผู้ใช้ทำการเรียกใช้งานซึ่งมีลักษณะดังนี้

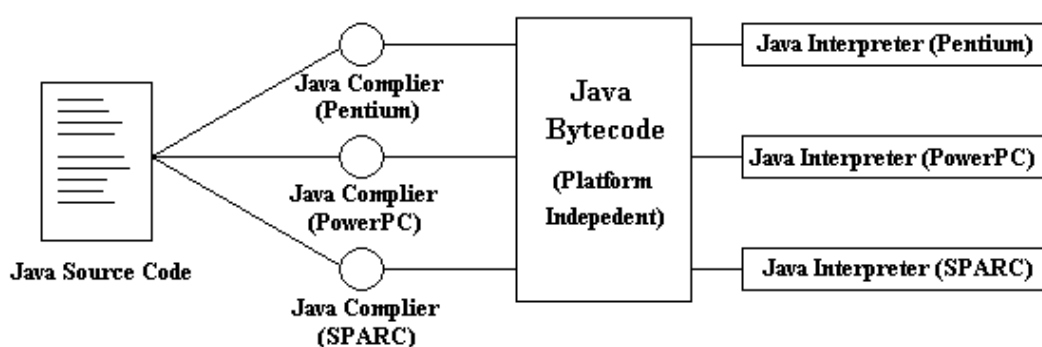
3.5.1 การคอมไพล์

คอมไพเลอร์ของภาษาจาวาก็เป็นเช่นเดียวกับคอมไพเลอร์ในภาษาอื่นๆ นั่นคือจะต้องสร้างรหัสภาษาเครื่อง (Machine Code หรือ Assembler Code) จากภาษาในระดับที่สูงกว่าเพื่อให้ซีพียูสามารถนำไปใช้งานได้ แต่ข้อแตกต่างที่สำคัญระหว่างคอมไพเลอร์ของภาษาจาวากับภาษาอื่นๆ คือไพเรสเซอร์หรือซีพียูที่จะคอยทำหน้าที่ในการปฏิบัติตามคำสั่งที่ได้จากการคอมไพล์ภาษาจาวานั้นไม่มีอยู่จริง เป็นเพียงสิ่งที่สมมติขึ้นมาที่เรียกว่า Java Virtual Machine นอกจากนี้การอ้างถึงส่วนต่างๆ ของโปรแกรมที่คอมไพล์ด้วยคอมไพเลอร์ของภาษาจาวาก็จะมีวิธีการที่แตกต่างออกไป

คอมไพเลอร์ของภาษาจาวาจะไม่เปลี่ยนการอ้างถึงส่วนของโปรแกรม จากการใช้ชื่อแบบในภาษาสูง ไปเป็นตัวเลขเหมือนที่คอมไพเลอร์ในภาษาอื่นๆ ทำกัน และคอมไพเลอร์ภาษาจาวาก็จะไม่มีการสร้างแผนที่ของการจัดวางโปรแกรมบนหน่วยความจำขึ้นมาในระหว่างการคอมไพล์ ด้วยเหตุผลที่สำคัญคือเพื่อเป็นการสร้างความพอร์ตเทเบิลให้กับตัวโปรแกรม เพราะการจัดวางตำแหน่งของโปรแกรมจะต้องขึ้นอยู่กับลักษณะการทำงานของไพเรสเซอร์ตัวใดตัวหนึ่ง การยังไม่จัดวางตำแหน่งช่วยให้โปรแกรมที่ได้จากการคอมไพล์มีความเป็นกลาง สามารถนำไปใช้บนคอมพิวเตอร์แพลตฟอร์มอื่นๆ ได้นอกจากนั้นยังทำให้เกิดความปลอดภัยอีกด้วย ซึ่งสิ่งที่ได้จากการคอมไพล์ในภาษาจาวาก็คือไบต์โค้ด



รูปที่ 3.2 การประมวลผลของโปรแกรมทั่วไปในระบบปฏิบัติการที่แตกต่างกัน



รูปที่ 3.3 การประมวลผลของโปรแกรมภาษาจาวา

3.5.2 การวางตำแหน่งในหน่วยความจำ

ในภาษาจาวาจะไม่มีกรลดรูปแบบการอ้างถึงส่วนต่างๆ ของโปรแกรมจากการเรียกเป็นชื่อให้เหลือเพียงตัวเลขหรือแอดเดรสที่กำหนดขึ้นจากการจัดวางตำแหน่งลงในหน่วยความจำ คอมไพเลอร์ของภาษาจาวาจะทิ้งชื่อของแต่ละส่วนของโปรแกรม (โดยเฉพาะเมธอด) เอาไว้ในตัวโปรแกรมที่สร้างขึ้นเมื่อโปรแกรมทำงานจะเป็นหน้าที่ของตัวอินเทอร์พรีเตอร์ ซึ่งจะคอยเปิดตารางค้นหาที่อยู่ของเมธอดที่ต้องการเรียกใช้งาน โดยก่อนที่จะเริ่มทำงานจริงอินเทอร์พรีเตอร์จะต้องสร้างแผนที่ในการจัดวางสิ่งต่างๆ ลงในหน่วยความจำขึ้นมาก่อน แล้วจึงสร้างตารางขึ้นมาเพื่อช่วยหาดำแหน่งของเมธอดเมื่อมีการเรียกใช้งานโดยใช้ชื่อของเมธอด

3.5.3 การรันโค้ดที่ได้จากการคอมไพล์

การรันโค้ดที่คอมไพล์เอาไว้สำหรับ Java Virtual Machine เป็นหน้าที่ของตัวอินเทอร์พรีเตอร์ การรันโปรแกรมจะแบ่งได้เป็น 3 ขั้นตอนหลักๆ คือ การอ่าน การตรวจสอบความถูกต้อง และการทำตามโค้ด หน้าที่ในการอ่านโค้ดเข้าสู่ระบบจะเป็นของคลาสโหลดเดอร์ (Class Loader) หน้าที่การทำงานในส่วนนี้จะไม่ได้อ่านเข้ามาเฉพาะๆ ไฟล์จาวาที่กำลังจะเรียกใช้เท่านั้น แต่จะอ่านคลาสที่มีการอ้างถึงและคลาสที่มีการสืบทอดต่อมาโดยคลาสที่อ้างถึง เมื่อผ่านขั้นตอนนี้แล้วโค้ดทั้งหมดก็就会被ส่งผ่านตัว

ตรวจสอบไบต์โค้ดเพื่อให้แน่ใจว่าโค้ดที่ส่งมามีความถูกต้องตามมาตรฐานของจาวาและจะไม่รบกวนเสถียรภาพของระบบ เมื่อผ่านการตรวจสอบแล้วโค้ดก็จะถูกส่งต่อไปยังระบบรันไทม์ (Run-Time System) ซึ่งจะส่งงานไปยังฮาร์ดแวร์อีกต่อหนึ่ง

3.5.4 คลาสโหลดเดอร์

คลาสโหลดเดอร์จะทำหน้าที่ดึงโค้ดทั้งหมดที่จำเป็นในการทำงานของแอปพลิเคชัน ไม่ว่าจะเป็นคลาสที่ถูกสืบทอดมาหรือคลาสอื่นๆ ที่มีการเรียกใช้เมื่อคลาสโหลดเดอร์ดึงคลาสใดเข้ามาแล้วก็จะจัดคลาสนั้นๆ ใส่เข้าไปใน namespace ของมันเองโดยการเก็บจะใช้ชื่อของคลาสเป็นสำคัญ ไม่ได้ใช้การอ้างถึงเป็นตัวเลข หลักการนี้จะเหมือนกับการทำงานของ Virtual Machine ในระบบปฏิบัติการที่สร้างขึ้นให้แอปพลิเคชันแต่ละตัวทำงาน ถ้าไม่ได้มีการเจาะจงเรียกใช้คลาสที่อยู่นอก namespace นี้ การเรียกใช้ชื่อต่างๆ ในคลาสก็จะไม่มีการรบกวนกันระหว่างคลาสเลย

คลาสทั้งหมดที่อยู่บนเครื่องโลคอล (local) จะได้รับช่องว่างแอดเดรส (Address Space) เป็นของตนเอง ส่วนคลาสต่างๆ ที่ดึงมาจากภายนอกจะได้รับ namespace เป็นของตนเอง การทำงานลักษณะนี้จะช่วยให้คลาสที่อยู่บนโลคอลทำงานได้ประสิทธิภาพดีขึ้นเพราะใช้ namespace ร่วมกันได้ แต่ก็ยังมีการป้องกันความผิดพลาดที่อาจเกิดจากคลาสที่ดึงเข้ามาจากภายนอก และในทางกลับกันคลาสที่นำเข้ามาที่ปลอดภัยจากความผิดพลาดที่อาจเกิดขึ้นจากคลาสโลคอลด้วย

เมื่อคลาสทั้งหมดที่เกี่ยวข้องกับการทำงานถูกนำเข้ามาเรียบร้อยแล้ว การจัดวางหน่วยความจำสำหรับเริ่มต้นการทำงานก็จะเกิดขึ้นได้การเรียกชื่อต่างๆ จะสามารถจับคู่กับแอดเดรสจริงๆ ของหน่วยความจำได้ แล้วตัวโหลดเดอร์จะสร้างตารางสำหรับค้นหาที่อยู่เลยจากการทำงานผิดปกติของซูเปอร์คลาส (Super class) และการอ้างแอดเดรสที่ไม่ถูกต้องได้

3.5.5 การตรวจสอบไบต์โค้ด

เมื่อโค้ดเดินทางมาจนถึงขั้นตอนการสร้างตารางจับคู่ชื่อกับแอดเดรสแล้ว ก็ยังไม่สามารถแน่ใจได้ว่าโค้ดที่อ่านเข้ามาจะมีความปลอดภัยดังนั้นจึงต้องมีตัว Verifier หรือตัวตรวจสอบไบต์โค้ดทำหน้าที่ตรวจสอบความถูกต้องที่ละบรรทัดว่าเป็นไปตามข้อกำหนดของจาวา และ สอดคล้องกับการทำงานของตัวโปรแกรมเองหรือไม่ การตรวจสอบโค้ดในเชิงทฤษฎีจะสร้างและค้นหาปัญหาต่างๆ ได้หลายอย่างเช่น จะไม่มีการสร้างพอยต์เตอร์ที่เกินกว่าหน่วยความจำจริง ไม่มีคำสั่งใดสามารถละเมิดสิทธิ์การทำงานของตัวโปรแกรมได้ ไม่มีการจับคู่วัตถุผิด ไม่มีการให้โอเพอร์เรนด์มากหรือน้อยเกินไป การกำหนดค่าต่างๆ สำหรับไบต์โค้ดจะต้องถูกต้องครบถ้วนและจะไม่มีการแปลงข้อมูลผิดรูปแบบ

การใช้ตัวตรวจสอบตอบสนองจุดประสงค์ 2 ประการที่สำคัญคือ สิ่งต่างๆ ดังที่กล่าวมาแล้วจะถูกตรวจสอบก่อน ทำให้ตัวอินเทอร์พรีเตอร์มั่นใจได้ว่าไบต์โค้ดที่ส่งเข้าไปทำงานจะไม่มีขั้นตอนการทำงานที่สร้างปัญหาให้กับตัวระบบ และจุดประสงค์ที่สองก็คือตัวอินเทอร์พรีเตอร์จะทำงานตามไบต์โค้ดได้รวดเร็วกว่า เพราะไม่ต้องคอยระวังว่าจะมีปัญหาก่อขึ้นและไม่ต้องหยุดเป็นช่วงๆ เมื่อพบปัญหาและ

ต้องแก้ไข การทำงานไบต์โค้ดจะถูกตรวจสอบเพียงครั้งเดียวเท่านั้นและจะทำงานไปได้ตลอด ไม่ต้องมีการตรวจสอบซ้ำอีกเมื่อมีการเรียกกลับมาทำงานที่ส่วนเดิมของโปรแกรม

3.5.6 การทำงานตามโค้ด

เมื่อคลาสโหลดเคอร์ได้รวบรวมโค้ดเข้ามาสู่ระบบทำการจัดวางในหน่วยความจำ และตัวตรวจสอบได้ทำการตรวจสอบความถูกต้องแล้ว โค้ดก็จะถูกส่งต่อไปยังตัวอินเทอร์พรีเตอร์เพื่อทำงานตามคำสั่ง การทำงานตามคำสั่งของโค้ดก็คือการเปลี่ยนโค้ดให้กลายเป็นคำสั่ง การทำงานจริงที่ตัวระบบไคลเอนต์ที่รันโค้ดนี้สามารถทำงานได้ ซึ่งวิธีการที่ทำได้ก็มีอยู่ 2 วิธีด้วยกันคือ ตัวอินเทอร์พรีเตอร์ทำการคอมไพล์โค้ดเหล่านี้ให้กลายเป็นเน็ตฟโค้ดที่ตัวเครื่องไคลเอนต์เข้าใจแล้วค่อยทำงาน เพื่อให้ได้ความเร็วสูงสุดในการทำงาน อีกวิธีหนึ่งก็คือตัวอินเทอร์พรีเตอร์อ่านโค้ดเข้ามาแล้วตีความทำงานไปทีละคำสั่งและทำการตีความไปเรื่อยๆ ตลอดเวลาที่มีการทำงาน

โดยปกติแล้วผู้สร้างตัวอินเทอร์พรีเตอร์มักจะเลือกใช้วิธีการที่สอง รูปแบบของไบต์โค้ดในภาษาจาวามีความยืดหยุ่นเพียงพอที่จะสามารถเปลี่ยนไปทำงานบนเครื่องไคลเอนต์แบบต่างๆ ได้โดยไม่มีการก่อให้เกิดโอเวอร์เฮดมากเกินไป อย่างไรก็ตามไคลเอนต์ของจาวาบางระบบจะมีความสามารถในการทำงานได้ทั้งสองวิธีคือ โปรแกรมเมอร์สามารถจะเลือกใช้วิธีการคอมไพล์กับงานที่เน้นการคำนวณมากๆ เพื่อเป็นการเพิ่มสมรรถนะในการทำงานให้ได้เต็มที่ ซึ่งไคลเอนต์แบบนี้จะให้ทั้งความพอร์เทเบิลและสมรรถนะที่ดี

การสร้างระบบรันไทม์ที่ดีจะต้องถ่วงดุลความสำคัญ 3 ประการให้พอเหมาะ นั่นคือความพอร์เทเบิล ความปลอดภัยและสมรรถนะเรื่องของความพอร์เทเบิล ซึ่งทำได้โดยการใช้รูปแบบของไบต์โค้ดที่มีความเป็นกลางเพียงพอ สามารถนำไปรันบนเครื่องคอมพิวเตอร์แบบอื่นๆ ได้ง่าย นอกจากนั้นการที่ตัวอินเทอร์พรีเตอร์ทำการกำหนดการจัดวางตำแหน่งในหน่วยความจำในช่วงรันไทม์ แทนที่จะเป็นระหว่างการคอมไพล์เหมือนภาษาอื่นๆ ก็เป็นการเพิ่มความแน่นอนว่าคลาสต่างๆ ที่นำเข้ามาจะยังคงใช้ได้อยู่ตลอดเวลา ในเรื่องของความปลอดภัยนั้นเป็นสิ่งที่มีการคำนึงถึงอยู่ตลอดการทำงานของระบบรันไทม์ โดยเฉพาะในส่วนของการตรวจสอบไบต์โค้ดที่ทำให้แน่ใจได้ว่าโปรแกรมจะทำงานได้ถูกต้อง ส่วนเรื่องของสมรรถนะนั้นก็สามารถจัดการได้ในสองระยะคือ พยายามเอาโอเวอร์เฮดทั้งหลายไปใส่ไว้ที่ตอนเริ่มต้นโหลดโปรแกรมเข้ามาสู่ระบบหรือไม่ก็กำหนดให้ทำงานเป็นแบบแบ็กกราวนด์ (Back-Ground Thread) ด้วยสิ่งต่างๆ เหล่านี้ทำให้จาวาสามารถปล่อยสมรรถนะระดับที่น่าพอใจ โดยยังคงไว้ซึ่งความพอร์เทเบิลและสภาพแวดล้อมที่ปลอดภัย นอกจากนั้นยังสามารถจะดึงสมรรถนะระดับสูงสุดออกมาใช้ได้ทันทีเมื่อต้องการ

3.5.7 การสร้างและรันภาษาจาวา

การสร้างโปรแกรมจากจาวาก็เหมือนกับภาษาโปรแกรมอื่นๆ คือเขียนโค้ดโปรแกรมจากอิดิเตอร์ใดๆ ก็ได้บันทึกอยู่ในนามสกุล .java จากนั้นจึงนำไปคอมไพล์ด้วยคอมไพเลอร์ของจาวาจะได้ไฟล์ใหม่ในนามสกุล .class ที่เก็บรหัสของการคอมไพล์ไว้จาวาเรียกชื่อรูปแบบข้อมูลที่อยู่ในไฟล์ใหม่นี้ว่าไบต์โค้ด ซึ่ง

ไฟล์ที่ได้คือแอปเพล็ตนั่นเอง แอปเพล็ตยังไม่สามารถรันได้ทันทีเหมือนไฟล์นามสกุล .exe หรือ .com ที่เรารู้จักกัน เพราะข้อมูลแบบไบต์โค้ดจะมีรูปแบบข้อมูลที่อยู่ที่กลางระหว่างโค้ดโปรแกรม (Source Code) กับโค้ดที่คอมพิวเตอร์อ่านแล้วนำไปปฏิบัติงานได้ทันที (Machine Code) หากจะรันแอปเพล็ตบนระบบใดๆ จะต้องใช้อินเตอร์พรีเตอร์จาวาของระบบนั้นๆ เพื่อตรวจสอบความถูกต้องของไบต์โค้ดแล้วแปลให้เป็นรหัสภาษาเครื่องส่งให้ระบบปฏิบัติการนำไปรันต่อไปขั้นตอนการทำงาน

3.6 Java Foundation Classes

โดยปกติชุดพัฒนาภาษาจาวาจะมีส่วนสนับสนุนการจัดการและเครื่องมือต่างๆ สำหรับสร้างส่วนติดต่อกับผู้ใช้งาน (User Interface) อยู่ก่อนแล้วคือ AWT (Abstract Window Toolkit) แต่ต่อมาทางบริษัท ซัน ไมโครซิสเต็มส์ ได้ร่วมมือกับหน่วยงานอื่นๆ คือ เน็ตสเคป, ไอบีเอ็มและ Lighthouse Design เพื่อทำการปรับปรุงประสิทธิภาพของส่วนติดต่อกับผู้ใช้งานให้มีความหลากหลายและใช้งานง่าย โดยสิ่งที่ถูกเพิ่มเข้ามาใหม่นี้ถูกเรียกว่าชุด JFC (Java Foundation Classes) ซึ่งถูกรวมอยู่กับ JDK เวอร์ชัน 1.1.5 ขึ้นไป แต่ที่เป็นมาตรฐานในการใช้งานจะอยู่ในรูปของ JDK เวอร์ชัน 1.2 โดยทางบริษัท ซัน ไมโครซิสเต็มส์ ได้เรียกชื่อใหม่ว่า Java 2

JFC เป็นชุดคำสั่งซึ่งแบ่งลักษณะการใช้งานออกเป็น 5 ชุดคำสั่งย่อยๆ ประกอบด้วย AWT API, Swing API, Java 2D API, Accessibility API และ Drag and Drop API ซึ่งทั้ง 5 ชุดคำสั่งย่อยนี้เกี่ยวข้องกับรูปแบบของการสร้างส่วนติดต่อกับผู้ใช้งาน

3.6.1 ชุดคำสั่ง AWT

AWT เป็นชุดคำสั่งที่บรรจุคอมโพเนนต์ต่างๆ สำหรับติดต่อกับผู้ใช้งานทั้งในส่วนการรับข้อมูลและแสดงผล ซึ่งชุดคำสั่ง AWT ถูกบรรจุอยู่ใน JDK ตั้งแต่เวอร์ชันแรกและขยายคอมโพเนนต์ให้มีความสามารถและความหลากหลายในการใช้งานมากขึ้น โดยลักษณะการทำงานของชุดคำสั่ง AWT เป็นแบบ Heavyweight ซึ่งจะมีผลในการนำไปใช้งานบนแพลตฟอร์มที่แตกต่างกัน ตัวอย่างเช่นการใช้งานบนวินโดวส์กับยูนิกซ์จะมีการแสดงรูปแบบของวัตถุ (Object) แตกต่างกัน

3.6.2 ชุดคำสั่ง Swing

สวิงเป็นชุดคำสั่งที่บรรจุคอมโพเนนต์ต่างๆ สำหรับติดต่อกับผู้ใช้งานเช่นเดียวกับ AWT แต่มีรูปแบบการใช้งานที่หลากหลายมากขึ้น โดยจัดเป็นคอมโพเนนต์แบบ Lightweight ทำให้การแสดงผลบนจอภาพเหมือนกันแม้ว่าจะนำไปใช้งานบนแพลตฟอร์มที่ต่างกัน ทำให้หากมีผู้ใช้งานโปรแกรมไปใช้งานบนวินโดวส์หรือยูนิกซ์จะมีลักษณะการแสดงผลรูปแบบของวัตถุที่เหมือนกัน

3.6.3 ชุดคำสั่ง Java 2D

ชุดคำสั่งจาวา 2 มิติเป็นชุดคำสั่งที่เพิ่มเข้ามาใหม่ตั้งแต่ JDK เวอร์ชัน 1.2 ขึ้นไป โดยรวบรวมคลาสที่เกี่ยวข้องกับการจัดการภาพและตัวอักษรในรูปแบบ 2 มิติ ซึ่งเป็นผลมาจากการพัฒนาร่วมกัน

ระหว่างบริษัท ซัน ไมโครซิสเต็มส์ และ IBM Taligent โดยตั้งความหวังว่าชุดคำสั่งจาวา 2 มิติจะช่วยให้การจัดการแสดงภาพง่ายขึ้นและสามารถนำไปสร้างโปรแกรมสำหรับการนำเสนอได้อย่างสะดวก

3.6.4 ชุดคำสั่ง Accessibility

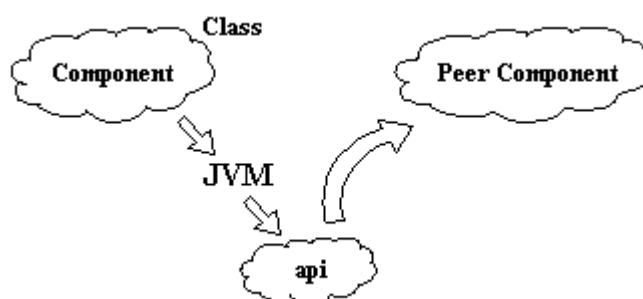
ชุดคำสั่ง Accessibility คือชุดคำสั่งที่ใช้สร้างส่วนติดต่อสำหรับผู้ที่ทุพพลภาพให้สามารถใช้โปรแกรมได้ ซึ่งสิ่งนี้เป็นสิ่งที่บริษัท ซัน ไมโครซิสเต็มส์ ได้เล็งเห็นว่าจะสามารถช่วยให้โปรแกรมเมอร์สร้างโปรแกรมเพื่อให้ผู้ทุพพลภาพไม่ว่าจะเป็นความพิการทางด้านใดสามารถใช้งานคอมพิวเตอร์ได้ ความสามารถของชุดคำสั่ง Accessibility มีมากมาย ตัวอย่างเช่นชุดคำสั่งสำหรับอ่านออกเสียงข้อมูลบนหน้าจอ (Screen Reader) และชุดคำสั่งจดจำเสียงสนทนา (Speech Recognition) เป็นต้น

3.6.5 ชุดคำสั่ง Drag and Drop

ชุดคำสั่ง Drag และ Drop เป็นชุดคำสั่งซึ่งสนับสนุนการทำงานโดยการลากและปล่อยเมาส์ โดยส่วนสำหรับติดต่อกับผู้ใช้งานจะสังเกตเหตุการณ์ที่ผู้ใช้งานทำการเลือกชิ้นส่วนที่ต้องการ โดยวิธีการลากเมาส์ และนำไปวางไว้บนพื้นที่ใดๆ ซึ่งเราพอสังเกตพฤติกรรมการใช้งานเหล่านี้ได้จากโปรแกรมต่างๆ เช่น PowerPoint และ Visio เป็นต้น

3.6.6 ความแตกต่างระหว่างชุดคำสั่ง AWT และชุดคำสั่ง Swing

หลายคนอาจสงสัยทำไมชุดคำสั่ง AWT จึงขึ้นอยู่กับแพลตฟอร์มต่างๆ ที่โปรแกรมภาษาจาวานั้นไม่ขึ้นกับแพลตฟอร์มใดๆ เหตุผลที่เป็นเช่นนี้เนื่องจากอินเทอร์พรีเตอร์ของภาษาจาวาจะต้องเป็นของแพลตฟอร์มใดแพลตฟอร์มหนึ่งเท่านั้น เพราะต้องเปลี่ยนไบต์โค้ดเป็นคำสั่งของแพลตฟอร์มนั้น เมื่อโปรแกรมของคลาส AWT ถูกคอมไพล์เป็นไบต์โค้ดแล้วโปรแกรมไบต์โค้ดที่ได้จะไม่ขึ้นกับแพลตฟอร์ม แต่ช่วงที่ถูกทำงานโดยอินเทอร์พรีเตอร์จะถูกเปลี่ยนเป็นชุดคำสั่งของแพลตฟอร์มนั้น แล้วทำการวาดและควบคุมการทำงานของตัวคอมโพเนนต์



รูปที่ 3.4 การทำงานของชุดคำสั่ง AWT

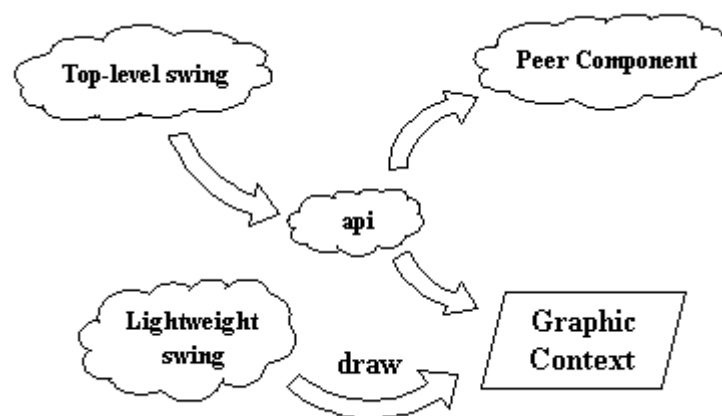
ซึ่งจากวิธีดังกล่าวมีข้อเสียคือโปรแกรมที่ใช้คลาสของ AWT ตัวอย่างเช่น Button หากนำไปทำงานบน win32 จะได้ button แบบของวินโดวส์ แต่ในโปรแกรมเดียวกันหากนำไปทำงานบนโซลาริส

จะได้ button แบบโมทีฟ (Motif) ทำให้คอมโพเนนต์หนึ่งเมื่อนำไปใช้งานบนแพลตฟอร์มที่ต่างกันจะได้รูปร่างที่แตกต่างกันและที่แย่ไปกว่านั้นคือบางคอมโพเนนต์อาจมีพฤติกรรมไม่เหมือนกันเมื่ออยู่ต่างแพลตฟอร์ม ด้วยเหตุนี้ทำให้โปรแกรมที่ใช้งานชุดคำสั่ง AWT ต้องขึ้นกับแพลตฟอร์ม

ข้อเสียอีกประการหนึ่งคือ ไบต์โค้ดของคอมโพเนนต์หนึ่งเมื่อถูกทำงานจะต้องเปลี่ยนไปเป็นคำสั่งจำนวนมากเพื่อวาดและควบคุมตัวคอมโพเนนต์นั้น เพราะคำสั่งหนึ่งมักจะทำงานที่ค่อนข้างพื้นฐานมาก การติดต่อเรียกทำงานระหว่างสภาพแวดล้อมเสมือน (Java Virtual Machine) กับคำสั่งรวมทั้งการนำส่งเหตุการณ์ (event) ให้แก่กันทำได้ค่อนข้างช้า มีโอเวอร์เฮดสูง เราจึงเรียกคอมโพเนนต์ประเภทนี้ว่า heavyweight ซึ่งหมายถึงคอมโพเนนต์ที่ต้องอาศัยคำสั่งของแพลตฟอร์มนั้น ซึ่งกราฟิกของส่วนติดต่อกับผู้ใช้งานในชุดคำสั่ง AWT ทุกตัวเป็นคอมโพเนนต์ heavyweight

ทางแก้หนึ่งของปัญหาดังกล่าวคือ การใช้คอมโพเนนต์ lightweight ซึ่งหมายถึงคอมโพเนนต์ที่วาดตัวเองและจัดการเหตุการณ์ของตัวเองด้วยภาษาจาวาโดยไม่ต้องใช้คำสั่งของแพลตฟอร์มนั้นเลย จึงทำให้รูปร่างและพฤติกรรมของคอมโพเนนต์ไม่ขึ้นกับแพลตฟอร์ม รวมทั้งปัญหาเกี่ยวกับโอเวอร์เฮดของคำสั่งก็หมดไป

แม้ว่ากราฟิกของส่วนติดต่อกับผู้ใช้งานในสวิงจะไม่ใช้คอมโพเนนต์ lightweight ทั้งหมดเนื่องจากคอมโพเนนต์ lightweight ไม่สามารถใช้งานวาดขึ้นบนจอภาพหรืออุปกรณ์รองรับกราฟิกได้โดยตรง แต่จะสามารถวาดลงบนพื้นที่กราฟิกของคอมโพเนนต์ตัวหนึ่งที่เราสร้างไว้ให้ก่อนแล้วเท่านั้น เราเรียกคอมโพเนนต์ที่มีกราฟิกสำหรับให้คอมโพเนนต์ตัวอื่นวาดว่าคอมโพเนนต์ top-level swing



รูปที่ 3.5 การทำงานร่วมกันของคอมโพเนนต์ top-level swing กับคอมโพเนนต์ lightweight

จะสังเกตได้ว่าคอมโพเนนต์ lightweight ของสวิงคือกลุ่มที่ไม่ใช่คอมโพเนนต์ top-level swing โดยในชุดคำสั่งสวิงมีเพียงบางตัวเป็นคอมโพเนนต์ heavyweight ทำหน้าที่เป็นคอมโพเนนต์ top-level swing ให้คอมโพเนนต์ lightweight ตัวอื่นๆ วาดใส่ตัวมัน ดังนั้นในการใช้งานแล้วเราต้องมีคอมโพเนนต์ top-level swing อย่างน้อยตัวหนึ่งก่อนจะวาดคอมโพเนนต์ lightweight ลงไปได้

3.7 เครื่องมือสำหรับพัฒนาโปรแกรมภาษาจาวา

การศึกษาและพัฒนาโปรแกรมภาษาจาวาในช่วงเริ่มแรกนั้นมีลักษณะที่เรียกว่า command line driven compiler โดยเครื่องมือแรกที่ใช้ในการประยุกต์เพื่อสร้างโปรแกรมภาษาจาวาคือ Sun's Java Development Kit (JDK) ในปัจจุบันมีเครื่องมือที่ช่วยในการพัฒนาโปรแกรมภาษาจาวาซึ่งได้รวมเอาคุณสมบัติของการสร้างกราฟิกบนส่วนสำหรับติดต่อกับผู้ใช้งาน (Graphic User Interface) ไว้ในการพัฒนาจาวาแอปพลิเคชัน โดยเครื่องมือสำหรับพัฒนานี้จะต้องเข้ากับมาตรฐานของภาษาจาวา ซึ่งทำให้เครื่องมือที่ใช้พัฒนาโปรแกรมภาษาจาวามีความง่าย และสะดวกกว่าในการที่จะศึกษาและใช้งานภาษาจาวามากขึ้น เครื่องมือสำหรับพัฒนาโปรแกรมภาษาจาวานั้นมีหลายบริษัทที่พัฒนาเครื่องมือเหล่านั้นออกมา โดยตัวอย่างเครื่องมือสำหรับพัฒนาโปรแกรมภาษาจาวาที่รู้จักกันมีดังต่อไปนี้

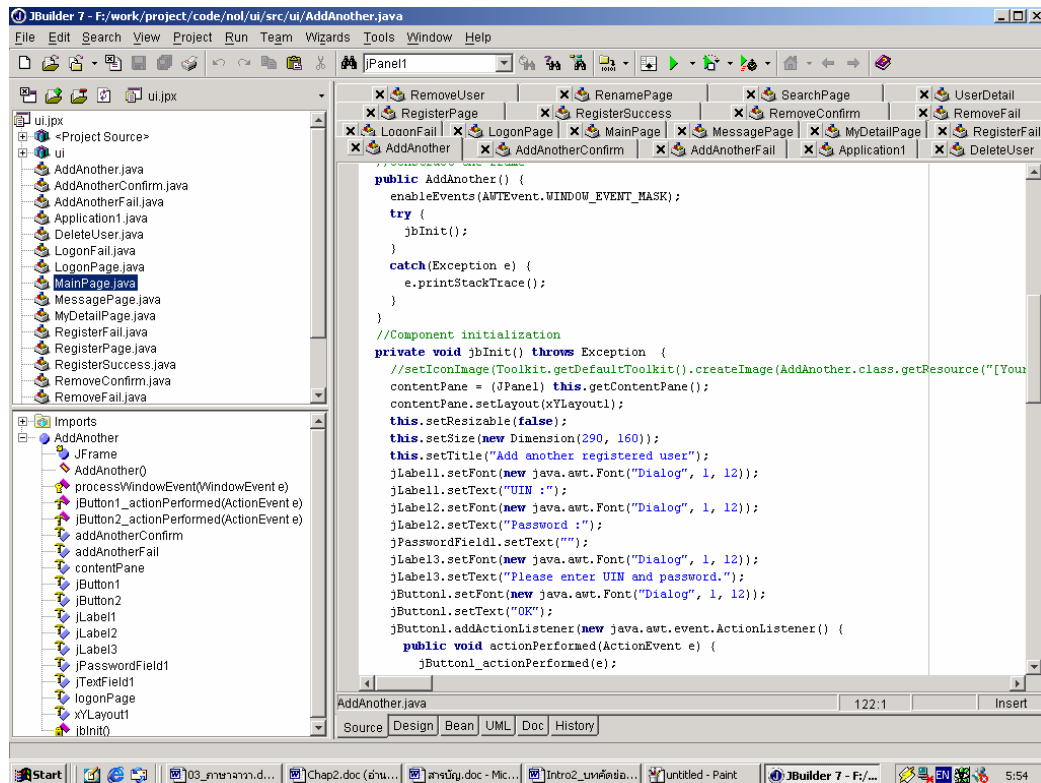
3.7.1 Java Development Kit

Java Development Kit หรือ JDK ถือเป็นเครื่องมือแรกของบริษัท ซัน ไมโครซิสเต็มส์ ที่ผลิตออกมาเป็นเครื่องมือสำหรับใช้ช่วยในการสร้างและพัฒนาโปรแกรมภาษาจาวา ซึ่งประกอบไปด้วยไฟล์ต่างๆ สำหรับใช้ในการทำงานดังต่อไปนี้

1. ไฟล์ appletviewer.exe ใช้สำหรับทดสอบจาวาแอปเพล็ต เพื่อดูผลการทำงานโดยไม่ต้องสั่งรันผ่านทางเว็บเบราว์เซอร์
2. ไฟล์ java.exe เป็นอินเทอร์พรีเตอร์สำหรับใช้ทดสอบการทำงานของจาวาแอปพลิเคชัน
3. ไฟล์ javac.exe มาจากคำว่าจาวาคอมไพเลอร์ใช้สำหรับแปลซอสโค้ดของโปรแกรมที่เราเขียนขึ้น (.java) ให้เป็นโปรแกรมไบต์โค้ด (.class)
4. ไฟล์ javap.exe ใช้แปลงคลาสไฟล์ของจาวาที่เป็นภาษาแอสเซมบลีกลับมาเป็นซอสโค้ด
5. ไฟล์ javadoc.exe ใช้สำหรับสร้างไฟล์เอกสารจากซอสโค้ดของจาวาในรูปแบบของ HTML ภายในจะเป็นการบอกรายละเอียดของคลาสต่างๆ ที่ผู้ใช้เขียนขึ้น กฎการเขียนที่ควรรู้ไว้กรณีที่มีผู้สนใจอยากจะสร้างคือ ผู้ใช้จะต้องใส่คำสั่ง /** เข้าไปด้วยเสมอ ตรงส่วนบนของโปรแกรมก่อนการประกาศคลาส ลักษณะการใช้งานจะเหมือนกับคำสั่ง // และ /* */ (สำหรับใส่ Comment คำอธิบายทั่วๆ ไป)

3.7.2 Borland JBuilder

Borland JBuilder เป็นเครื่องมือสำหรับพัฒนาโปรแกรมภาษาจาวาของบอร์แลนด์ ซอฟท์แวร์ คอร์ปอเรชัน (Borland Software Corporation) ซึ่ง JBuilder เป็นเครื่องมือที่สนับสนุนการทำงานบนระบบปฏิบัติการวินโดวส์, โซลาริส, ลินุกซ์และแมค จุดเด่นของ JBuilder ที่สำคัญคือสามารถใช้งานได้ง่าย สามารถตรวจสอบจุดบกพร่องได้ดี ซึ่งทำให้ JBuilder เป็นเครื่องมือที่เหมาะสมสำหรับงานสร้างและพัฒนาโปรแกรมขนาดใหญ่ นอกจากนี้ JBuilder ยังมียูทิลิตี้ (Utilities) สำหรับเพิ่มความสามารถให้กับโปรแกรมอีกมากมายเช่น Package Migration tool และ JDBC Explorer เป็นต้น



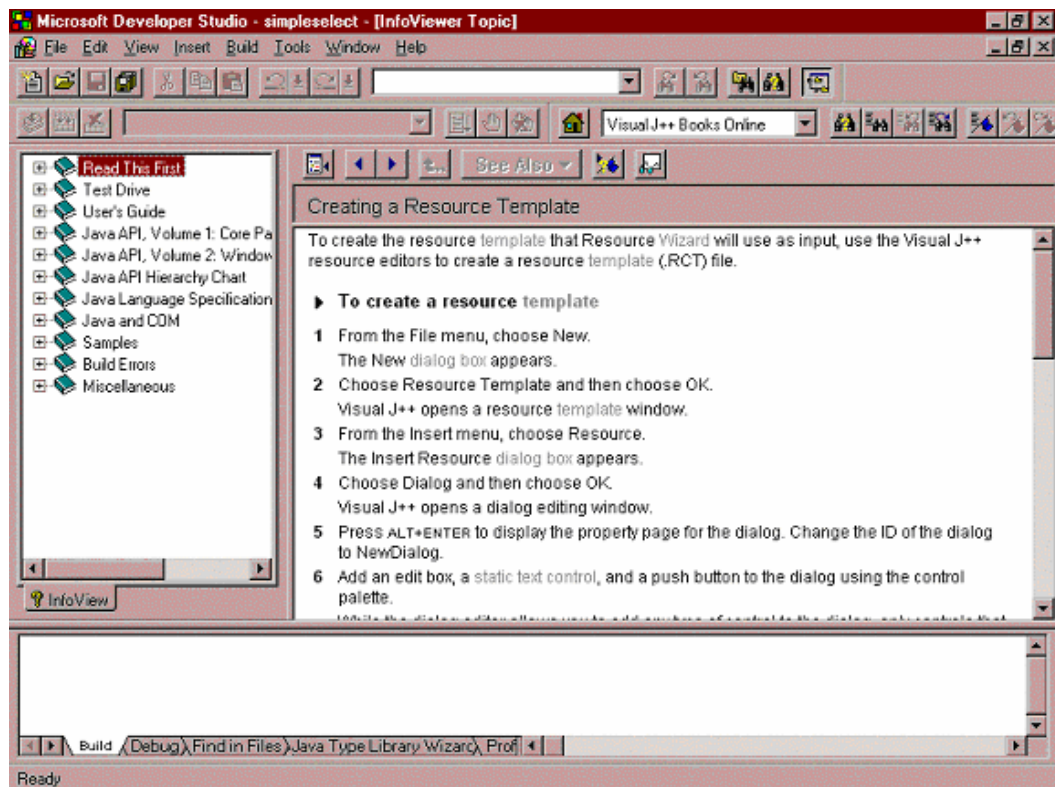
รูปที่ 3.6 โปรแกรม Borland JBuilder

หากดูจากภาพรวมแล้ว JBuilder ก็เป็นเครื่องมือในการพัฒนาโปรแกรมจาวาที่น่าใช้งานตัวหนึ่ง แต่ JBuilder ก็มีจุดอ่อนที่ต้องใช้หน่วยความจำมากและประมวลผลช้า นอกจากนี้โปรแกรม JBuilder สำหรับองค์กรธุรกิจก็ยังมีราคาสูงมาก

3.7.3 Microsoft Visual J++

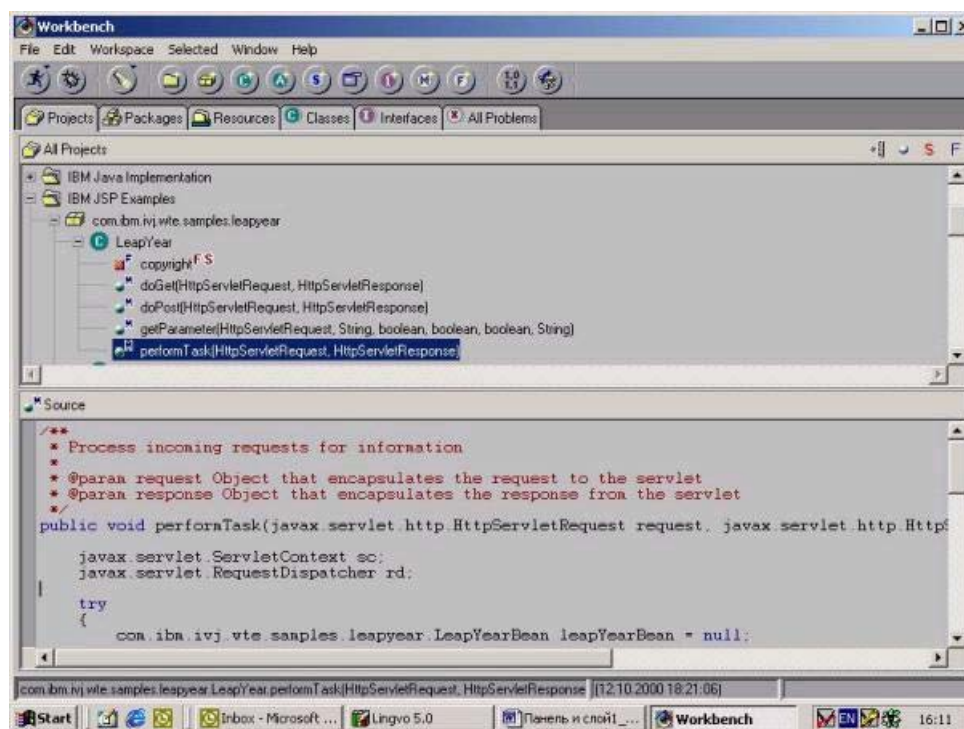
Visual J++ เป็นเครื่องมือที่ใช้พัฒนาการเขียนจาวาแอปเพล็ตและแอปพลิเคชันโดยไมโครซอฟท์ คอร์ปอเรชั่น ซึ่ง Visual J++ จัดเป็นเครื่องมือที่มีจุดเด่นในเรื่องการตรวจสอบจุดบกพร่องเหมือนกับ JBuilder แต่ไม่ใช้หน่วยความจำมากเหมือนกับ JBuilder นอกจากนี้ Visual J++ ยังมีความสามารถในการ visually create java forms โดย resource editor จะยอมให้ผู้ใช้สามารถใช้ visually layout ในการติดต่อกับ แอปพลิเคชันของผู้ใช้เอง ซึ่งผู้ใช้งานยังสามารถที่จะคอนเวิร์ตเป็นรูปแบบของ Visual Basic และ Visual C++

จากลักษณะของการคอนเวิร์ตใน Visual J++ จะพบว่า Visual J++ จะสนับสนุนการใช้งานร่วมกับเครื่องมือหรือคอมโพเนนต์ของไมโครซอฟท์ได้ดี ซึ่งในส่วนนี้เองทำให้ Visual J++ ไม่เข้ากับจาวาได้ดี (Java-compatible) เหมือนกับเครื่องมือพัฒนาโปรแกรมจาวาอื่นๆ



รูปที่ 3.7 โปรแกรม Microsoft Visual J++

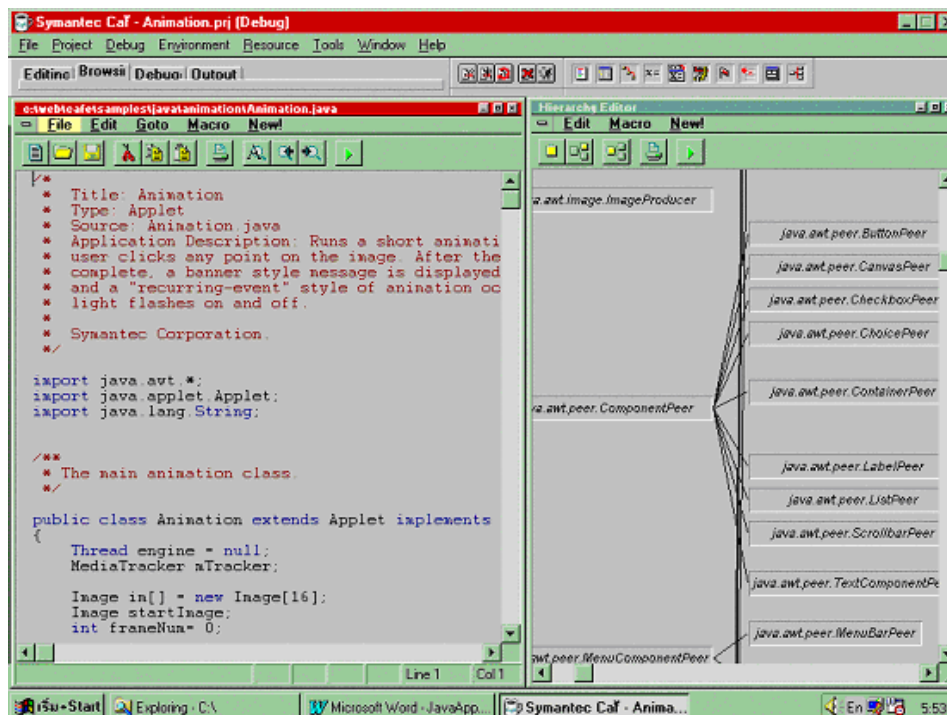
3.7.4 IBM VisualAge for Java



รูปที่ 3.8 โปรแกรม IBM VisualAge for Java

VisualAge เป็นเครื่องมือที่ใช้พัฒนาโปรแกรมจาวาของไอบีเอ็ม VisualAge จัดว่าเป็นเครื่องมือที่มีความน่าเชื่อถือสูง ตรวจสอบจุดบกพร่องได้ดี สามารถใช้งานได้ทั้งบนระบบปฏิบัติการวินโดวส์และลินุกซ์ แต่ข้อเสียของ VisualAge คือการใช้งานค่อนข้างยุ่งยาก อาจต้องใช้เวลาศึกษานาน เป็นโปรแกรมที่มีขนาดใหญ่ ใช้หน่วยความจำมากและประมวลผลช้า

3.7.5 VisualCafe



รูปที่ 3.9 โปรแกรม VisualCafe

VisualCafe เป็นเครื่องมือที่ถูกคิดค้นและพัฒนาโดย Symantec แต่ปัจจุบันถูกพัฒนาโดย Webgain ซึ่ง VisualCafe จัดเป็นเครื่องมือที่มีขนาดเล็ก ใช้หน่วยความจำไม่มาก สามารถทำงานได้รวดเร็ว และสนับสนุนการใช้งาน Java 2 สำหรับจุดบกพร่องของ VisualCafe ก็คือสามารถใช้งานได้กับระบบปฏิบัติการวินโดวส์เท่านั้นและเป็นเครื่องมือที่มีความน่าเชื่อถือต่ำ

3.8 ความแตกต่างระหว่างจาวาแอปพลิเคชันและจาวาแอปเพล็ต

จาวาแอปพลิเคชันและจาวาแอปเพล็ต ต่างเป็นภาษาคอมพิวเตอร์ซึ่งถูกออกแบบมาเพื่อใช้งานอินเทอร์เน็ตเหมือนกัน โดยทำงานอยู่บนโฮมเพจร่วมกับภาษา HTML เหมือนกัน แต่ก็มีลักษณะการนำไปใช้งานที่แตกต่างกันคือ

- จาวาแอปพลิเคชันสามารถปฏิบัติงานเดี่ยวๆ ได้ด้วยตัวเองเช่นเดียวกับการเขียนโปรแกรมด้วยภาษาอื่นๆ ทั่วไป แต่สิ่งที่ควรจดจำในการเขียนจาวาแอปพลิเคชันก็คือผู้เขียนโปรแกรมจะต้องมี

เมธอดของ `Main(string args[])` อย่างน้อยๆ 1 เมธอดสำหรับใช้เป็นจุดเริ่มต้นของโปรแกรม ซึ่งหากสังเกตให้ดีจะเหมือนกับการเขียนโปรแกรมโดยใช้ภาษาซี ส่วนวิธีการคอมไพล์โปรแกรมก็จะใช้ `javac.exe` ซึ่งจะทำให้ได้ไฟล์นามสกุล `.class` เพื่อใช้ในการแสดงผลของโปรแกรม โดยในส่วนการแสดงผลเราจะใช้โปรแกรม `java.exe` ซึ่งจะทำให้การแสดงผลของไฟล์นามสกุล `.class` ที่ได้มาจากการคอมไพล์ในข้างต้น

- จาวาแอปเพล็ตไม่สามารถทำงานเดี่ยวๆ ด้วยตนเองได้เหมือนกับจาวาแอปพลิเคชัน โดยที่จาวาแอปเพล็ตจะทำงานร่วมกันกับไฟล์ HTML ซึ่งเราจะเรียกใช้ไฟล์ที่เขียนจากจาวาแอปเพล็ตนี้ผ่านทางแท็กคำสั่ง `<APPLET>...</APPLET>` ภายในไฟล์ HTML จากนั้นก็จะเรียกใช้ไฟล์โปรแกรม `javac.exe` มาทำการคอมไพล์เช่นเดียวกับการใช้จาวาแอปพลิเคชันซึ่งจะได้ไฟล์ที่มีนามสกุล `.class` ไว้สำหรับให้ไฟล์ HTML เรียกผ่านวิธีแสดงผลการทำงานของจาวาแอปเพล็ต ซึ่งในการแสดงผลของจาวาแอปเพล็ตนั้นจะใช้โปรแกรม `appletviewer.exe` หรือเว็บเบราว์เซอร์ เช่น เน็ตสเคป, ฮอตจาวา, Internet Explorer และอื่นๆ โดยเลือกใช้ตัวใดตัวหนึ่งก็ได้

3.9 ทิศทางของจาวา

จาวาเริ่มต้นจากการเป็นภาษาคอมพิวเตอร์ง่ายๆ สำหรับสร้างแอปเพล็ตหรือโปรแกรมขนาดเล็กๆ ซึ่งสามารถทำงานบนเครื่องคอมพิวเตอร์ระบบใดๆ ก็ได้ โดยไม่จำเป็นต้องสร้างโปรแกรมที่ใหญ่โตมากนัก มาปัจจุบันนี้กระแสด้านความต้องการและความนิยมของจาวายังคงสูงขึ้นอย่างรวดเร็ว ส่งผลให้เกิดพัฒนาโปรแกรมประยุกต์หรือแอปพลิเคชันใหม่ๆ โดยใช้ภาษาจาวา ซึ่งจาวาไม่ได้เป็นเพียงแค่แอปเพล็ตเล็กๆ อีกต่อไปแล้วเพราะเราสามารถใช้งานจาวาเป็นเครื่องมือสำหรับสร้างซอฟต์แวร์ขนาดใหญ่

ในปี ค.ศ.1996 ถือเป็นปีแห่งการพัฒนาซอฟต์แวร์ประเภทไคลเอ็นต์ หลายต่อหลายฝ่ายต่างมุ่งเป้าความสนใจไปที่เครื่องคอมพิวเตอร์ประเภทโต้ตอบกับผู้ใช้ในฝั่งไคลเอ็นต์กับแอปพลิเคชันต่างๆ ที่สำหรับใช้ภายในองค์กร ต่อมาในปี ค.ศ.1997 เป็นปีที่ซอฟต์แวร์จะก้าวไปไกลมากกว่าไคลเอ็นต์ โดยจะเคลื่อนที่เข้าไปใกล้ชิดกับผู้ใช้งานยิ่งขึ้นในรูปแบบของอุปกรณ์ส่วนบุคคลต่างๆ โดยการใช้งานควบคุมอุปกรณ์ไฟฟ้าและเครื่องมือช่วยระบบดิจิทัลส่วนบุคคล, อุปกรณ์ Set-top Boxes (กล่องควบคุมอนกประสงค์) Smart Phones, ระบบฝังตัวในอุปกรณ์ต่างๆ เช่นในเครื่องพิมพ์, เครื่องส่งแฟกซ์, โทรศัพท์เว็บ, โทรศัพท์เว็บเบราว์เซอร์ รวมไปถึงเครื่องใช้ไฟฟ้าต่างๆ ซึ่งอุปกรณ์เหล่านี้จะถูกเชื่อมการติดต่อกับระบบ Smart Cards นั้นหมายถึงว่าผู้บริโภคสามารถที่จะตั้งโปรแกรมควบคุมระยะไกลกับสวิตช์ไฟฟ้าผ่านทางอุปกรณ์ช่วยขนาดเล็กเหล่านี้

นอกจากนี้ในปัจจุบันจาวายังเข้ามามีบทบาทกับการใช้งานบนโทรศัพท์มือถือ PDA's และอุปกรณ์ไร้สาย โดยที่ผู้พัฒนาสามารถเขียนโปรแกรมเพื่อสร้างแอปพลิเคชันให้ทำงานบนอุปกรณ์เหล่านี้ได้เสมือนเป็นคอมพิวเตอร์ขนาดเล็กอีกเครื่องหนึ่ง ซึ่งเทคโนโลยีการเขียนโปรแกรมด้วยภาษาจาวาบนเครื่องมือสื่อสารเหล่านี้ก็คือ J2ME (Java 2 Micro Edition) ตัวอย่างของแอปพลิเคชันที่พัฒนามาจาก J2ME และนำมาใช้งานบนอุปกรณ์สื่อสารได้เช่น โปรแกรมวิเคราะห์และแสดงราคาหุ้น, โปรแกรมรับส่งไฟล์, โปรแกรมรับส่งสารด่วนและเกมออนไลน์ เป็นต้น ทำให้การใช้งานอุปกรณ์สื่อสารกลับมามีผู้ใช้

ความสนใจอย่างมากอีกครั้งหนึ่ง และจากการเติบโตของการใช้งานอุปกรณ์สื่อสารนี้ทางบริษัท ชัน ไมโครซิสเต็มส์ ร่วมกับ 3COM จึงพัฒนาอุปกรณ์ปาล์มท็อปที่ใช้เทคโนโลยีของจาวาออกมาด้วย เพื่อเป็นการชดเชยความสามารถของปาล์มท็อป เพราะปาล์มท็อปมีข้อจำกัดคือ พื้นที่ในการเก็บข้อมูลซึ่งโดยส่วนใหญ่ปาล์มท็อปจะมีพื้นที่สำหรับเก็บข้อมูลกัน 2-8 เมกะไบต์ (สามารถขยายได้ถึง 16 เมกะไบต์) ดังนั้นการจะเก็บตัวโปรแกรมและข้อมูลทุกอย่างไว้ในเครื่องนั้นย่อมเป็นไปได้ ดังนั้นจาวาจึงถือเป็นเครื่องมือหนึ่งที่เหมาะสมสำหรับพัฒนางานนี้

ซึ่งขณะนี้ยังคงมีนักประดิษฐ์จำนวนมากต่างพยายามที่จะสร้างสรรค์อุปกรณ์แปลกๆ ใหม่ๆ ขึ้นมาเพื่อตอบสนองความต้องการของมนุษย์ ซึ่งภาษาจาวายังคงเป็นเครื่องมือหนึ่งที่มีศักยภาพสูงสุดในการสนับสนุนอุปกรณ์ใหม่ๆ เหล่านี้